

Лабораторная работа №8

Целочисленная арифметика многократной точности

Кюнкриков Даниил Саналович (НПИМд-01-24, 1132249574)

Российский университет дружбы народов (РУДН), Москва

- Кюнкриков Даниил Саналович
- студент уч. группы НПИМд-01-24
- Российский университет дружбы народов
- 1132249574@pfur.ru
- репозиторий: https://github.com/DanzanK/2025-2026_math-sec/tree/main

В криптографии и теории чисел часто используются числа, не помещающиеся в стандартные типы данных, поэтому нужны поразрядные алгоритмы “длинной арифметики”: сложение, умножение деление и т.д.

Цель работы: программная реализация целочисленных операций многократной точности в основании b .

Задачи:

- Выбрать представление числа как массив цифр в основании b ;
- Реализовать алгоритмы 1-5 (сложение, вычитание, умножение неотрицательных чисел столбиком, умножение быстрым столбиком, деление многоразрядных целых чисел)
- проверить работу на тестовых примерах

- Основание системы счисления: $b \geq 2$,
- Число хранится как массив цифр:
 - 'dig[0]' - младший разряд,
 - 'dig[1]' - следующий разряд и т.д. Такое хранение упрощает перенос/заем

Для каждого разряда:

$$s = u_j + v_j + k, \quad w_j = s \bmod b, \quad k = \left\lfloor \frac{s}{b} \right\rfloor.$$

При $u \geq v$: - если $u_j - v_j - k < 0$, то прибавляем b и ставим $k = 1$, - иначе $k = 0$

Алгоритм 3 - Умножение столбиком

- Перебираем разряды второго множителя.
- Для каждого разряда выполняем умножение на все разряды первого с переносом.
- Частичные результаты суммируются в массив результата.

- Суммирование производится по диагоналям (по сумме индексов).
- Для каждой диагонали вычисляем сумму произведений и перенос, затем выделяем цифру $t \bmod b$.

Требуется найти:

$$u = qv + r, \quad 0 \leq r < v.$$

В реализации: - оценка очередной цифры частного по старшим разрядам; -
корректировка, если оценка завышена; - нормализация (подход Knuth D).

Реализация

Программная реализация алгоритма суммирования

```
def add_big(u: list[int], v: list[int], b: list[int]) -> list[int]:
    n = max(len(u), len(v))
    w = [0] * (n + 1)
    carry = 0
    for j in range(n):
        uj = u[j] if j < len(u) else 0
        vj = v[j] if j < len(v) else 0
        s = uj + vj + carry
        w[j] = s % b
        carry = s // b

    w[n] = carry
    return trim(w)
```

Рисунок 1: Алгоритм суммирования

Программная реализация алгоритма вычитания

```
def sub_big(u: list[int], v: list[int], b: int) -> list[int]:
    if cmp_digits(u,v) < 0:
        raise ValueError("Err u should be >= v")
    n = len(u)
    w = [0] * n
    borrow = 0
    for j in range(n):
        uj = u[j]
        vj = v[j] if j < len(v) else 0
        s = uj - vj - borrow
        if s < 0:
            s += b
            borrow = 1
        else:
            borrow = 0
        w[j] = s
    return trim(w)
```

Рисунок 2: Алгоритм вычитания

Программная реализация алгоритма умножения столбиком

```
def mul_classic(u: list[int], v: list[int], b: int) -> list[int]:
    u = trim(u[:])
    v = trim(v[:])
    if u == [0] or v == [0]:
        return [0]
    n, m = len(u), len(v)
    w = [0] * (n+m)
    for j in range(m):
        if v[j] == 0:
            continue
        carry = 0
        for i in range(n):
            t = u[i] * v[j] + w[i + j] + carry
            w[i + j] = t % b
            carry = t // b
        w[j + n] += carry
    carry = 0
    for k in range(len(w)):
        t = w[k] + carry
        w[k] = t % b
        carry = t // b
    while carry > 0:
        w.append(carry % b)
        carry //= b
    return trim(w)
```

Рисунок 3: Алгоритм умножения столбиком

Программная реализация алгоритма умножения быстрым столбиком

```
def mul_fast(u: list[int], v: list[int], b: int) -> list[int]:
    u = trim(u[:])
    v = trim(v[:])
    if u == [0] or v == [0]:
        return [0]
    n, m = len(u), len(v)
    w = [0] * (n+m)
    carry = 0
    for s in range(n + m - 1):
        t = carry
        i_min = max(0, s - (m-1))
        i_max = min(n - 1, s)
        for i in range(i_min, i_max + 1):
            t += u[i] * v[s-i]
        w[s] = t % b
        carry = t // b
    w[n + m - 1] = carry
    k = n + m - 1
    while w[k] >= b:
        extra = w[k] // b
        w[k] %= b
        k += 1
        if k == len(w):
            w.append(0)
        w[k] += extra

    return trim(w)
```

Рисунок 4: Алгоритм умножения быстрым столбиком

Программная реализация алгоритма деления многоразрядных целых чисел

```
def div_big(u: list[int], v: list[int], b: int) -> tuple[list[int], list[int]]:
    u = trim(u[:])
    v = trim(v[:])

    if v == [0]:
        raise ZeroDivisionError("Err")
    if u == [0]:
        return [0], [0]
    if cmp_digits(u,v) < 0:
        return [0], u
    if v == [1]:
        return u, [0]
    U = to_be(u)
    V = to_be(v)
    n = len(V)
    m = len(U) - n
    uu = [0] + U[:]
    vv = V[:]
    d = b // (vv[0] + 1)
    if d > 1:
        mul_digit_be_inplace(uu, d, b)
        mul_digit_be_inplace(vv, d, b)

    n = len(vv)
    m = len(uu) - n - 1
    q = [0] * (m+1)

    v0 = vv[0]
    v1 = vv[1] if n > 1 else 0
```

Рисунок 5: Алгоритм деления многоразрядных целых чисел

```
for j in range(m + 1):
    u0 = uu[j]
    u1 = uu[j + 1]
    numer = u0 * b + u1
    qhat = numer // v0
    rhat = numer % v0
    if qhat >= b:
        qhat = b - 1
        rhat += v0
    if n > 1:
        u2 = uu[j + 2]
        while qhat * v1 > rhat * b + u2:
            qhat -= 1
            rhat += v0
            if rhat >= b:
                break

    borrow = 0
    for i in range(n-1, -1, -1):
        p = qhat * vv[i] + borrow
        idx = j + i + 1
        t = uu[idx] - (p % b)
        borrow = p // b
        if t < 0:
            t += b
            borrow += 1
        uu[idx] = t
    t0 = uu[j] - borrow
    if t0 < 0:
        qhat -= 1
        t0 += b
        carry = 0
```

Рисунок 6

```
for i in range(n - 1, -1, -1):
    idx = j + i + 1
    t = uu[idx] + vv[i] + carry
    if t >= b:
        t -= b
        carry = 1
    else:
        carry = 0
    uu[idx] = t
    uu[j] = t0 + carry
    if uu[j] >= b:
        uu[j] -= b
    else:
        uu[j] = t0
    q[j] = qhat

r = uu[m + 1 :]
if d > 1:
    r = div_digit_be(r,d,b)

q_le = to_le(q)
r_le = to_le(r)
return q_le, r_le
```

Рисунок 7


```
0 - exit
1 - сложение
2 - вычитание
3 - умножение столбиком (классическое)
4 - умножение быстрым столбиком
5 - деление с остатком
```

```
Выберите алгоритм (0 для выхода): 1
Введите основание b (2-36)4
Введите u: 3
Введите v: 3
u + v = 12
```

```
Выберите алгоритм (0 для выхода): 2
Введите основание b (2-36)4
Введите u: 3
Введите v: 3
u - v = 0
```

```
Выберите алгоритм (0 для выхода): 3
Введите основание b (2-36)4
Введите u: 3
Введите v: 3
u * v = 21
```

```
Выберите алгоритм (0 для выхода): 4
Введите основание b (2-36)4
Введите u: 3
Введите v: 3
u * v = 21
```

```
Выберите алгоритм (0 для выхода): 5
Введите основание b (2-36)4
Введите u: 21
Введите v: 3
q = 3
r = 0
```

```
Выберите алгоритм (0 для выхода):
```

- Реализованы основные операции длинной арифметики с задаваемым основанием b
- Показана корректность на тестовых примерах

Данный подход является базой для криптографических и численных алгоритмов, где требуются большие целые числа